ChessLatro

Technical Overview — How the Project Works

Project Overview

ChessLatro is a fully playable chess game built with the **Godot 4.4** game engine. It supports all standard chess rules and uses the **Stockfish** chess engine as an AI opponent. The game is written in **GDScript** for game logic, with a **C#** wrapper to interface with Stockfish. The player controls White; Black is handled automatically by Stockfish at search depth 10.

Tech Stack

Component	Technology
Game Engine	Godot 4.4
Primary Language	GDScript
AI Bridge	C# (.NET 8.0)
Chess AI	Stockfish (external process)
Rendering	Forward Plus renderer
Art Assets	PNG sprites (alpha theme)

Architecture & File Structure

The project is split into six GDScript files and one C# file, each with a clear responsibility:

File	Role
scripts/game_state_manager.gd	Autoloaded singleton — owns all board state and move validation
scripts/chess_board.gd	Main scene — draws the 8x8 board and routes tile click events
scripts/movement_manager.gd	Handles user input, executes moves, triggers AI, manages turn flow
scripts/piece_manager.gd	Places starting pieces and creates piece sprites from PNG assets
scripts/promotion_popup.gd	Pawn promotion UI — emits signal with chosen piece type
scripts/camera_controller.gd	Scales the viewport/camera to fit the board responsively
StockfishInterface.cs	C# class — spawns Stockfish process, sends FEN, returns best move

1. Game State Manager (game_state_manager.gd)

This is the central brain of the game, registered as a Godot **Autoload singleton** called **GameStateManager**. It is accessible from every other script without needing a reference.

What it stores

- board_state — an 8x8 array of piece codes (e.g. 'wp', 'bk', '' for empty)

- current_turn — 'w' or 'b'
- Castling rights — four booleans (white/black x kingside/queenside)
- en_passant_target — Vector2i of the square a pawn can capture via en passant
- move_log — array of all moves played

Piece Code Format

Pieces are identified by a two-character string: **[color][type]**

```
'w' = white, 'b' = black
```

```
'p'=pawn 'n'=knight 'b'=bishop 'r'=rook 'q'=queen 'k'=king
```

```
Examples: 'wp' = white pawn, 'bk' = black king, 'wq' = white queen
```

Move Validation

Each piece type has its own validation function:

- is_valid_pawn_move — handles forward moves, double-step from start rank, diagonal captures, and en passant
- is_valid_knight_move — checks L-shaped offsets (2+1 or 1+2)
- is_valid_bishop_move — walks diagonals, fails if any piece is in the way
- is_valid_rook_move — walks ranks/files, fails if any piece blocks the path
- is_valid_queen_move — delegates to bishop + rook validators
- is_valid_king_move — one square any direction, plus castling (2-square move with rook)

Check, Checkmate & Stalemate

is_king_in_check scans every opponent piece and tests whether it can attack the king's square. has_legal_move simulates every possible move for a color, temporarily applies it to board_state, checks if the king is still in check, then reverts — returning True if any safe move exists. Checkmate = in check + no legal moves. Stalemate = not in check + no legal moves.

FEN Generation

board_state_to_fen converts the internal board array into a FEN string, which is the universal chess position format used to communicate with Stockfish. It encodes piece positions, active color, castling rights, and en passant target square.

2. Chess Board (chess_board.gd)

The main scene script. On startup it calls draw_board() to create 64 ColorRect nodes (one per square), naming each one with its algebraic coordinate (e.g. 'e4', 'h8'). Each tile is 80x80 pixels, colored white or gray based on (rank + file) % 2. Tile click signals are connected here and forwarded to MovementManager.

3. Movement Manager (movement_manager.gd)

Handles all interactive gameplay. The main flow on a tile click:

- First click — if the tile has a piece belonging to the current turn, select it (highlight gold)
- Second click — validate the move using GameStateManager validators + is_move_safe (king safety check)
- If valid — update board_state, move the sprite visually, handle special cases
- Call end_turn() — evaluate board state, switch current_turn, trigger AI if it is Black's turn

Special Move Handling

- Castling — detected when king moves 2 squares; rook is also moved in board_state and visually
- En passant — captured pawn is removed from its actual square (not the destination square)
- Promotion — pawn reaching rank 1 (white) or rank 8 (black) triggers the promotion popup

is_move_safe

Before committing any move, the manager simulates it on board_state, calls is_king_in_check, then reverts. This prevents players from making moves that leave their own king in check.

4. Stockfish Integration (StockfishInterface.cs)

A C# class decorated with [GlobalClass] so Godot treats it as a native node. When it is Black's turn, MovementManager calls GetBestMove(fen):

- Spawns a new Stockfish process with stdin/stdout redirected
- Sends: position fen
- Sends: go depth 10 (searches 10 moves deep)
- Reads output lines until one starts with 'bestmove'
- Parses and returns the move string (e.g. 'e7e5', 'e8g8' for castling, 'e7e8q' for promotion)
- Sends 'quit' and waits for the process to exit

The returned move string is in UCI format (from-square + to-square + optional promotion piece). apply_stockfish_move in MovementManager parses this string and applies it to the board.

5. Piece Manager (piece_manager.gd)

Handles visual representation of pieces. place_starting_pieces sets up the standard chess starting position by placing PNG sprites on tiles and updating GameStateManager's board_state. create_piece_sprite loads the correct PNG from res://assets/pieces/alpha/.png and returns a scaled TextureRect sized to fit the 80px tile.

6. Turn Flow Diagram

A complete turn cycle from player click to AI response:

Step	What Happens
1. Player clicks a tile	chess_board.gd routes to movement_manager.handle_tile_click()
2. Piece selected	Tile highlighted gold; waiting for destination click
3. Destination clicked	Validators check move legality + king safety

4. Move applied	board_state updated, sprite moved, special cases handled
5. end_turn() called	evaluate_board_state() checks check/checkmate/stalemate
6. Turn switches to Black	call_deferred('_make_ai_move') scheduled
7. FEN generated	board_state_to_fen() produces current position string
8. Stockfish called	GetBestMove(fen) runs Stockfish at depth 10
9. AI move applied	apply_stockfish_move() parses UCI string, updates board
10. end_turn() called again	evaluates state, switches back to White

7. Notes & Known Quirks

- Checkmate/stalemate is detected and printed to console but there is no in-game UI for it yet
- Stockfish spawns a new process on every move — no persistent engine session
- Black castling validation is kept as redundant safety even though Stockfish handles it
- The promotion popup blocks all input while visible (mouse_filter = IGNORE on the board)
- piece_manager.place_starting_pieces has several commented-out debug position variants
- move_log is tracked but not currently used for display or replay

---